

Relatório de Desenvolvimento: Quiz em Tempo Real (Estilo Kahoot)

Disciplina: Redes de Computadores I — PUC Minas

Alunos:

- Bruno Menezes Rodrigues Oliveira Vaz
- João Costa Calazans
- João Pedro Torres
- Lucas Carneiro Nassau Malta
- Pedro Henrique Debs Rabelo

1 Objetivo do Trabalho

O objetivo deste trabalho prático é desenvolver e implantar uma aplicação de rede do tipo **Quiz em Tempo Real** (estilo *Kahoot*), utilizando a arquitetura **Cliente-Servidor** sob os protocolos **TCP** e **UDP** com **multithreading** em Java. O projeto também abrange a simulação e configuração de uma rede hierárquica contendo três roteadores no software **Cisco Packet Tracer**, demandando regras de roteamento estático e redirecionamento de portas (NAT/PAT) para viabilizar a comunicação fim-a-fim, além da validação de tráfego por meio do analisador de protocolos **Wireshark**.

2 Metodologia e Arquitetura

A aplicação foi desenvolvida em Java 17, dividida em dois submódulos independentes e sem dependências externas (`client/` e `server/`), permitindo compilação e execução isoladas.

2.1 Canal de Controle e Jogabilidade (TCP)

- **Finalidade:** Trafegar mensagens críticas do jogo de forma ordenada, livre de erros e confiável.
- **Mecanismo:** O servidor inicializa um `ServerSocket` escutando na porta **TCP 12345**. Os clientes se conectam e a comunicação ocorre por meio de streams de serialização de objetos (`ObjectOutputStream` e `ObjectInputStream`) trocando instâncias da classe comum `GameMessage`.
- **Mensagens Trafegadas:** Solicitação de login (`LOGIN`), confirmação de conexão (`LOGIN_RESPONSE`), envio de perguntas e alternativas (`QUESTION`), envio da resposta do jogador (`ANSWER`), resultados parciais/scoreboard (`ROUND_RESULT`) e sinalização de encerramento (`GAME_OVER`).

2.2 Canal do Cronômetro (UDP)

- **Finalidade:** Transmissão periódica em tempo real e de baixa latência dos segundos restantes da rodada.
- **Mecanismo:** No ato de conexão (via payload do pacote TCP de `LOGIN`), o cliente informa uma porta UDP local livre (geralmente a **12346**). O servidor instancia uma thread dedicada (`QuizTimer.java`) que a cada segundo envia pacotes UDP unicast (`DatagramSocket`) contendo o payload de texto `"TIMER;<segundos>"` diretamente ao endereço IP e à porta UDP cadastrada de cada cliente ativo.

2.3 Multithreading e Interface Gráfica (Swing)

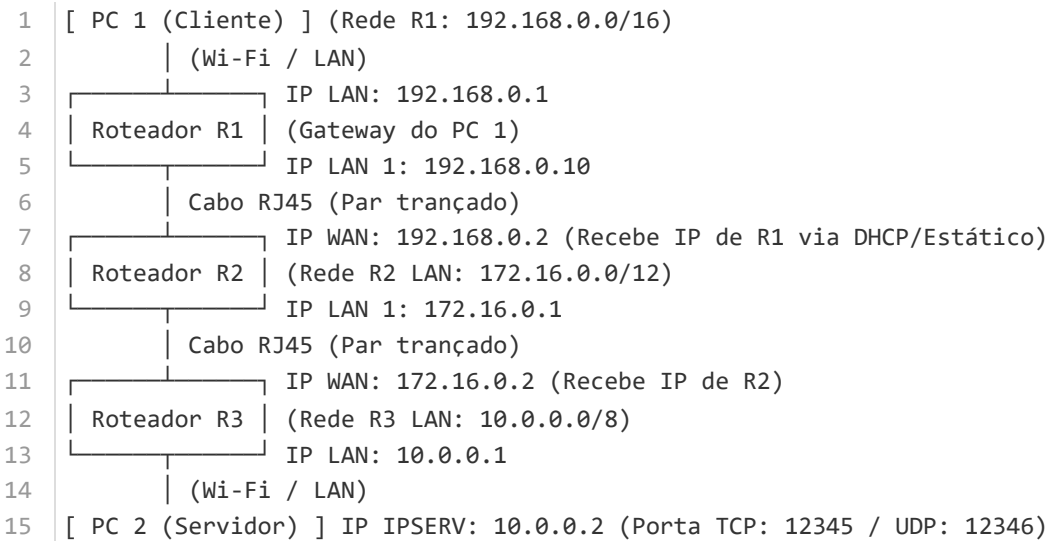
Tanto o cliente (`QuizWindow`) quanto o servidor (`ServerWindow`) utilizam Java Swing sob o padrão *Dracula Palette*. A execução baseia-se em concorrência para manter a interface responsiva:

- **No Servidor:** A thread principal inicia a GUI no *Event Dispatch Thread* (EDT). Uma thread paralela escuta novas conexões TCP. Para cada cliente conectado, é gerada uma thread `ClientHandler.java` responsável pela leitura de mensagens de rede. Durante a rodada, a contagem do tempo é controlada por uma thread temporária `QuizTimer.java`.
- **No Cliente:** A thread principal gerencia a transição de telas via `CardLayout`. Threads secundárias executam a escuta contínua de pacotes TCP (`startTCPReceiver`) e UDP (`startUDPReceiver`), atualizando os componentes de interface de maneira assíncrona por meio do `SwingUtilities.invokeLater()`.

3 Modelagem e Configuração da Rede (Cisco Packet Tracer)

A rede simula um ambiente corporativo/residencial com três roteadores ligados em cascata e mascaramento NAT nas bordas.

3.1 Topologia Física e Lógica (Esquema ASCII)



3.2 Tabela de Endereçamento IP

Dispositivo/Interface	Rede Lógica	Endereço IP / Máscara	Gateway Padrão
PC 1 (Cliente)	Rede R1	192.168.0.100 /16	192.168.0.1
Roteador R1 (LAN)	Rede R1	192.168.0.1 /16	N/A
Roteador R2 (WAN)	Rede R1	192.168.0.2 /16	192.168.0.1
Roteador R2 (LAN)	Rede R2	172.16.0.1 /12	N/A
Roteador R3 (WAN)	Rede R2	172.16.0.2 /12	172.16.0.1
Roteador R3 (LAN)	Rede R3	10.0.0.1 /8	N/A
PC 2 (Servidor)	Rede R3	10.0.0.2 /8	10.0.0.1

3.3 Configuração de Roteamento e Redirecionamento (NAT / Port Forwarding)

Para que o PC 1 (na rede 192.168.0.0/16) consiga conectar no servidor (PC 2, na rede interna 10.0.0.2), configurou-se uma cadeia de redirecionamentos nos roteadores. O cliente direciona a conexão para o IP do próximo salto visível.

1. Roteador R3 (Borda do Servidor):

- Configuração de NAT *Inside/Outside* nas interfaces LAN (`inside`) e WAN (`outside`).
- Regra de Port Forwarding (PAT): Redireciona conexões recebidas na interface WAN (porta TCP `12345`) para o IP interno do Servidor.
 - Comando CLI: `ip nat inside source static tcp 10.0.0.2 12345 172.16.0.2 12345`

- Redirecionamento UDP: `ip nat inside source static udp 10.0.0.2 12346 172.16.0.2 12346`

2. Roteador R2 (Roteador Intermediário):

- Redireciona conexões recebidas na interface WAN (porta TCP 12345 e UDP 12346) para o IP da interface WAN de R3.
 - Comando CLI: `ip nat inside source static tcp 172.16.0.2 12345 192.168.0.2 12345`
 - Redirecionamento UDP: `ip nat inside source static udp 172.16.0.2 12346 192.168.0.2 12346`

3. Roteador R1 (Borda do Cliente):

- Redireciona conexões destinadas ao seu próprio IP local (se o cliente apontar para o gateway) ou apenas roteia os pacotes diretamente. Como o PC 1 e a WAN de R2 estão no mesmo domínio de difusão de R1 (192.168.0.0/16), o PC 1 pode iniciar a conexão TCP apontando diretamente para o IP 192.168.0.2 (WAN de R2) na porta 12345, percorrendo o túnel NAT de forma transparente até o PC 2.

4 Avaliação de Tráfego (Wireshark)

Abaixo, descreve-se o comportamento do fluxo de rede capturado durante a execução de uma partida típica:

4.1 Estabelecimento da Conexão (TCP Three-Way Handshake)

- **Captura:** O PC 1 (192.168.0.100) envia um segmento TCP com a flag SYN ativada para o IP da WAN do R2 (192.168.0.2) na porta 12345.
- **Tradução:** R2 recebe e redireciona para R3 WAN (172.16.0.2), que por sua vez traduz a porta de destino para o IP do PC 2 (10.0.0.2).
- **Acordo:** O PC 2 responde com SYN-ACK. O pacote percorre o caminho reverso, sofrendo as devidas traduções de cabeçalho NAT pelos roteadores R3 e R2. O PC 1 finaliza o handshake enviando o ACK. A conexão lógica TCP está estabelecida.

4.2 Fluxo da Partida e Protocolo de Aplicação

1. **Login e Registro UDP:** O cliente envia um pacote TCP contendo um objeto serializado do tipo GameMessage (Type=LOGIN, payload="Jogador1", value=12346). O servidor lê o objeto e associa o socket TCP do cliente ao envio do timer na porta UDP 12346.
2. **Ciclo da Pergunta e Timer:**
 - O servidor transmite via TCP a mensagem Type=QUESTION com o texto e as alternativas. A interface gráfica do cliente é atualizada e os botões de resposta são habilitados.
 - Imediatamente, a thread QuizTimer do servidor passa a emitir datagramas UDP unicast de 1 em 1 segundo contendo "TIMER;15", "TIMER;14", etc., para 192.168.0.100:12346 (IP do cliente visto após a reversão de NAT pelos roteadores).
3. **Respostas e Interrupção Antecipada:** O jogador clica na alternativa (ex: "C"). O cliente envia uma mensagem TCP Type=ANSWER, payload="C". O servidor registra o tempo da resposta para fins de pontuação.
 - **Otimização:** Quando todos os jogadores respondem, o servidor chama o método checkAllAnswered(), interrompendo (interrupt()) a thread QuizTimer imediatamente, economizando banda e tempo ao encerrar a rodada sem aguardar a expiração dos 15 segundos padrão.
4. **Resultados e Finalização:** O servidor envia via TCP o ROUND_RESULT individual e o scoreboard acumulado. Ao fim de 4 perguntas, o servidor transmite GAME_OVER, exibe a classificação final e encerra a conexão TCP transmitindo pacotes com a flag FIN ativada, que são respondidos com ACK pelos clientes, efetuando o encerramento limpo da sessão de rede.

5 Principais Desafios e Soluções

5.1 Concorrência e Conflitos com a EDT do Java Swing

- **Desafio:** Modificações na interface Swing a partir de threads de rede receptoras secundárias frequentemente travavam a interface (*deadlocks*) ou geravam comportamento inconsistente.
- **Solução:** Toda e qualquer alteração de estado dos componentes visuais do cliente e do servidor (como transição de cartões no `CardLayout`, atualização do texto do cronômetro ou renderização de rankings) foi encapsulada em chamadas assíncronas seguras via `SwingUtilities.invokeLater()`.

5.2 Roteamento de Retorno UDP com Cadeia de NAT

- **Desafio:** Datagramas UDP são protocolos sem conexão e sem estado. Se os roteadores fizessem tradução NAT restritiva pura em múltiplas camadas para conexões de saída UDP, os pacotes do cronômetro enviados de forma espontânea pelo servidor (de dentro para fora) poderiam ser bloqueados ou não encontrar a rota correta para o cliente sem um mapeamento ativo prévio.
- **Solução:** Na simulação do Packet Tracer, utilizou-se o redirecionamento estático bidirecional (regras estáticas de NAT/PAT para UDP na porta `12346`), garantindo que qualquer datagrama UDP endereçado à porta do cliente fosse roteado e traduzido corretamente ao atravessar as sub-redes em cascata.